

[Insert K L University Logo Here]

**KONERU LAKSHMAIAH EDUCATION FOUNDATION
K L (DEEMED TO BE) UNIVERSITY**

Department of Computer Science and Engineering

A PROJECT REPORT ON

LINUX RESOURCE MONITORING TOOL

(Official title: Linux System Information and Resource Monitoring Tool)

Submitted in partial fulfilment of the requirements of the course
OPERATING SYSTEMS AND SYSTEMS PROGRAMMING

Submitted by

S. No.	Name of the Student	Roll Number
1	B. Sathish Kumar	2520090008
2	P. Rahul	2520090016
3	Ch. Sai Vihamsh	2520090090

Under the Guidance of
Mr. Raghupathi Manthena

Section: 10 Team: 03
Term-I Academic Year: 2026–27

DECLARATION

We, the undersigned students of the Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, K L (Deemed to be) University, hereby declare that the project report entitled "**Linux Resource Monitoring Tool**" (submitted as "Linux System Information and Resource Monitoring Tool") is an original work carried out by us under the guidance of **Mr. Raghupathi Manthena**, as part of the course **Operating Systems and Systems Programming** during Term-I of the Academic Year 2026–27 (Section 10, Team 03).

We further declare that this work has not been submitted, in part or in full, to any other university or institution for the award of any degree, diploma or certificate. All sources of information, tools and documentation that were referred to during the project have been duly acknowledged in the references.

The project has been developed in C++ with a GTK4 graphical interface, and it was developed and tested in Ubuntu through WSL. The contents of this report are a true record of the work carried out by the team.

Place: _____

Date: _____

B. Sathish Kumar
Roll No.: 2520090008

P. Rahul
Roll No.: 2520090016

Ch. Sai Vihamsh
Roll No.: 2520090090

CERTIFICATE

This is to certify that the project report entitled "**Linux Resource Monitoring Tool**" (official title: "Linux System Information and Resource Monitoring Tool") is a bonafide record of the work carried out by the following students of the Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, K L (Deemed to be) University, in partial fulfilment of the requirements of the course **Operating Systems and Systems Programming**, Term-I, Academic Year 2026–27, Section 10, Team 03.

Submitted by

B. Sathish Kumar – 2520090008

P. Rahul – 2520090016

Ch. Sai Vihamsh – 2520090090

Under the guidance of

Mr. Raghupathi Manthena

The work reported in this document has been carried out under our supervision, and to the best of our knowledge it has not been submitted to any other university or institution for any degree or diploma.

Guide Signature

Mr. Raghupathi Manthena

**Course Coordinator
Signature**

HOD Signature

Department of Computer
Science and Engineering

Place: _____

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Mr. Raghupathi Manthena**, for his valuable guidance, constant encouragement and timely suggestions throughout the development of the project "Linux Resource Monitoring Tool". His advice helped us to understand the practical side of Operating Systems concepts and to complete the project successfully.

We are thankful to the faculty and the course coordinator of **Operating Systems and Systems Programming** for providing us with the opportunity to carry out this project, and for explaining the concepts of processes, memory management, file systems and system programming that formed the basis of our work.

We extend our thanks to the **Department of Computer Science and Engineering** and to **Koneru Lakshmaiah Education Foundation, K L (Deemed to be) University**, for providing the academic environment, the laboratory facilities and the resources that were necessary for this project.

We also thank all the teaching and non-teaching staff of the department for their help and cooperation during the course of the work.

Finally, we are grateful to our parents, friends and classmates for their support, motivation and encouragement. We are also thankful to each other for the cooperation and teamwork that made it possible for us to complete this project.

B. Sathish Kumar (2520090008)
P. Rahul (2520090016)
Ch. Sai Vihamsh (2520090090)

ABSTRACT

Monitoring the use of system resources is an essential activity for anyone who uses, develops or administers a computer system. Processor time, memory, swap space, disk storage and running processes are shared by all programs, and when any of them becomes heavily loaded the performance of the whole system is affected. However, the information needed to understand this behaviour is normally available only as raw counters and text-based commands, which are difficult for beginners to read and interpret.

The Linux Resource Monitoring Tool, submitted as the "Linux System Information and Resource Monitoring Tool", is a desktop application developed in C++ with a GTK4 graphical user interface to present this information in a clear and organised manner. The tool collects data mainly from the Linux /proc virtual filesystem: /proc/stat for CPU statistics, /proc/meminfo for memory and swap information, and /proc/uptime for system uptime. It supplements these with Linux utilities, using ps for process information and df for disk usage, while the number of CPU cores is obtained with the sysconf() interface.

The dashboard displays CPU, memory, disk and swap usage, total memory, CPU cores, system uptime, the number of running processes, and the top CPU-consuming and top memory-consuming processes. A simple system health indicator interprets the usage values using fixed thresholds of 70% and 90%, and the time of the last update is shown. CPU Usage History and Memory Usage History graphs, drawn with Cairo/GTK drawing functionality, show how usage changes over time. The information refreshes automatically through a GTK4 timer and can also be updated with a Refresh button, and an About Project dialog provides information about the project.

The project demonstrates important Operating Systems concepts, namely CPU utilisation, memory management and swap, process management, the file system abstraction through the /proc virtual filesystem, and periodic resource monitoring. It was developed and tested in Ubuntu through WSL and compiled using g++, and it gives students a practical and easily understandable view of how a Linux system uses its resources.

Keywords: Linux, Operating Systems, Resource Monitoring, /proc Filesystem, GTK4, CPU Usage, Memory Usage, Process Monitoring, C++

INDIVIDUAL CONTRIBUTION

The work of the project was divided among the three team members as shown below.

Table 9.1: Individual Contribution of Team Members

Name and Roll Number	Responsibilities
B. Sathish Kumar 2520090008	• Project design • System information module • CPU and memory monitoring • Overall integration • GTK4 GUI integration
P. Rahul 2520090016	• Disk monitoring • /proc filesystem data collection • Testing • Documentation
Ch. Sai Vihamsh 2520090090	• Process monitoring • Process information collection • Top CPU consumer identification • Top memory consumer identification

TABLE OF CONTENTS

DECLARATION.....	i
CERTIFICATE.....	ii
ACKNOWLEDGEMENT	iii
ABSTRACT.....	iv
INDIVIDUAL CONTRIBUTION	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background of the Project.....	1
1.2 Problem Statement.....	1
1.3 Scope of the Project	2
1.4 Objectives of the Project.....	2
1.5 Importance of the Application.....	3
1.6 Target Users / Audience.....	3
CHAPTER 2 – OPERATING SYSTEM CONCEPTS AND BACKGROUND STUDY..	4
2.1 Operating System Concepts Related to the Project	4
2.2 CPU Management and CPU Utilization	4
2.3 Memory Management.....	5
2.4 Process Management	6
2.5 File Systems and Linux /proc.....	6
2.6 Resource Monitoring	7
2.7 Linux Commands and Utilities	7
2.8 Proposed Approach.....	8
2.9 Advantages of the Proposed Approach.....	9
CHAPTER 3 – SYSTEM REQUIREMENTS.....	10
3.1 Hardware Requirements.....	10
3.2 Software Requirements.....	10
3.3 Operating System Environment – Ubuntu / WSL.....	10
3.4 Development Tools and Compiler.....	11
CHAPTER 4 – TECHNOLOGY AND TOOLS USED	12
4.1 C++ Programming	12
4.2 GTK4	12
4.3 GCC / g++ Compiler	12
4.4 Linux /proc Virtual File System.....	12

4.5 Linux Commands and Utilities	13
4.6 Cairo / GTK Drawing	13
4.7 VS Code	13
4.8 WSL / Ubuntu	13
CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN.....	15
5.1 Overall System Architecture	15
5.2 System Workflow	16
5.3 Data Flow	17
5.4 Module Description	18
5.5 Input and Output Design.....	19
5.6 GUI Design	20
CHAPTER 6 – IMPLEMENTATION	22
6.1 Module-wise Implementation	22
6.2 CPU Monitoring Implementation.....	23
6.3 Memory Monitoring Implementation	24
6.4 Disk Monitoring Implementation	24
6.5 Swap Monitoring Implementation.....	24
6.6 Total Memory and CPU Core Information	25
6.7 System Uptime	25
6.8 Process Monitoring.....	25
6.9 Top CPU Consumer.....	26
6.10 Top Memory Consumer	26
6.11 System Health Calculation	27
6.12 CPU and Memory History Graphs	27
6.13 GTK4 GUI Implementation	28
6.14 Automatic Refresh.....	29
6.15 Refresh Button.....	29
6.16 About Project Module.....	29
CHAPTER 7 – EXECUTION AND DEMONSTRATION.....	30
7.1 Compilation in Ubuntu/WSL	30
7.2 Execution Commands	30
7.3 Sample Output.....	31
7.4 GUI Screenshots.....	32
7.5 Demonstration of OS Concepts.....	33
CHAPTER 8 – CONCLUSION	34
8.1 Summary of the Project	34
8.2 Objectives Achieved.....	34

8.3 Results Achieved	34
8.4 Skills and Learning Outcomes	35
8.5 Future Enhancements.....	35
CHAPTER 9 – REFERENCES	37

LIST OF FIGURES

Figure 5.1: Overall System Architecture	15
Figure 5.2: System Workflow	17
Figure 5.3: Data Flow Diagram	18
Figure 6.1: Linux Resource Monitoring Tool – Main Dashboard	23
Figure 6.2: CPU and Memory Usage History.....	28
Figure 7.1: Project Execution in Ubuntu/WSL	31
Figure 7.2: Compilation of the Project in Ubuntu/WSL	32
Figure 7.3: About Project Dialog	33

LIST OF TABLES

Table 1.1: Target Users of the Application.....	3
Table 2.1: Operating Systems Concepts Related to the Project.....	4
Table 2.2: Important Fields of /proc/meminfo	6
Table 2.3: /proc Files Used in the Project.....	7
Table 2.4: Interfaces and Utilities Used by the Project	8
Table 3.1: Hardware Requirements.....	10
Table 3.2: Software Requirements	10
Table 3.3: Development Tools.....	11
Table 4.1: Summary of Technologies Used.....	14
Table 5.1: Components of the System Architecture	16
Table 5.2: Module Description	19
Table 5.3: Input and Output Design	20
Table 5.4: Elements of the Main Dashboard	20
Table 6.1: Feature-wise Implementation Summary	22
Table 6.2: System Health Thresholds.....	27
Table 7.1: Explanation of the Compilation Command.....	30
Table 7.2: Representative Dashboard Values (Format Illustration).....	31
Table 7.3: Test Scenarios and Expected Behaviour	32
Table 7.4: Operating Systems Concepts Demonstrated by the Project	33
Table 8.1: Objectives and Their Achievement.....	34
Table 8.2: Possible Future Enhancements	36

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

An operating system is the system software that manages the hardware and software resources of a computer and provides common services to application programs. Every program that runs on a computer competes for a limited set of resources: processor time, main memory, swap space on secondary storage, and disk space. The operating system decides how these resources are shared, and the overall responsiveness of the computer depends on how well this sharing works.

When a computer becomes slow, an application stops responding, or a disk becomes full, the cause can usually be found by observing how the resources are being used. This activity is called resource monitoring. It is carried out by system administrators, software developers, and students who wish to understand how an operating system behaves while programs are executing.

Linux is one of the most widely used operating systems in servers, cloud platforms, embedded devices, and development environments. Linux exposes a large amount of information about the running system through the **/proc virtual filesystem**, which is a special area of the file hierarchy generated by the kernel in memory. In addition, standard Linux utilities such as `ps` and `df` report information about processes and disk usage. Although this information is freely available, it is presented as raw text and numeric counters, which are not easy for a beginner to read or interpret.

The **Linux Resource Monitoring Tool** developed in this project collects such information from the Linux system and presents it on a graphical dashboard built with GTK4. The tool is written in C++ and is intended to show, in a simple and visual manner, how the operating system's resource management concepts appear in a running system.

1.2 Problem Statement

The problem statement submitted for this project is titled "**Linux System Information and Resource Monitoring Tool**". It can be stated as follows:

To design and develop a Linux-based application that collects system information and resource usage details such as CPU usage, memory usage, disk usage, swap usage, system uptime and running processes using the Linux /proc filesystem and Linux utilities, and presents them to the user through an easy-to-understand graphical interface that refreshes automatically.

Existing command-line methods require the user to remember several commands, to read raw counters from different files, and to calculate percentages manually. For example, the CPU counters in `/proc/stat` increase continuously from system start-up and give no meaningful percentage unless two readings are compared. A student or a beginner therefore finds it difficult to understand the current state of the system. The project addresses this problem by automating the collection, calculation and presentation of resource information in one dashboard.

1.3 Scope of the Project

The scope of the current version of the Linux Resource Monitoring Tool is limited to **monitoring and displaying** system information. The following items are within the scope of the project:

- Monitoring of CPU usage, memory usage, disk usage and swap usage.
- Display of total memory, number of CPU cores, system uptime and the number of running processes.
- Identification and display of the top CPU-consuming process and the top memory-consuming process.
- A simple system health indication based on fixed usage thresholds.
- CPU Usage History and Memory Usage History graphs.
- Automatic periodic refresh, a manual Refresh button and an About Project dialog.
- A graphical user interface developed using GTK4 for Linux (Ubuntu), developed and tested through WSL.

The following items are **not** part of the current version and are listed only as possible future enhancements in Chapter 8: process control (terminating or changing the priority of processes), network monitoring, CPU temperature monitoring, storage of historical data, resource usage alerts, process filtering, and exporting of reports.

1.4 Objectives of the Project

The main objectives of the project are:

1. To study how Linux exposes CPU, memory, swap, uptime and process information through the `/proc` virtual filesystem and standard utilities.
2. To develop a C++ program that reads this information and calculates meaningful values such as CPU usage percentage and memory usage percentage.

3. To identify the processes that consume the highest CPU and the highest memory at a given moment.
4. To design a GTK4 graphical dashboard that presents the collected information clearly.
5. To provide CPU and memory usage history graphs so that the change in usage over time can be observed.
6. To implement automatic refreshing so that the dashboard always shows current information.
7. To provide a simple health indication that helps the user interpret the current resource condition.
8. To apply and demonstrate Operating Systems concepts such as CPU utilisation, memory management, process management and file systems in a practical project.

1.5 Importance of the Application

Resource monitoring is important because an operating system has finite resources and every running program affects them. Observing resource usage helps in detecting overloaded systems, identifying programs that consume excessive CPU or memory, planning storage capacity, and understanding system behaviour during program execution.

The importance of this application can be summarised as follows:

- **Educational value:** It connects theoretical Operating Systems concepts with real data produced by a running Linux system.
- **Ease of understanding:** Raw kernel counters are converted into percentages, graphs and simple status messages.
- **Single view:** CPU, memory, disk, swap, uptime and process information are available in one window instead of several commands.
- **Lightweight design:** The tool reads data directly from /proc and from two small utilities, and does not require any external monitoring library.
- **Foundation for extension:** The modular structure allows further monitoring features to be added in the future.

1.6 Target Users / Audience

Table 1.1: Target Users of the Application

User Group	How the Application Helps
Students	Helps in understanding CPU utilisation, memory usage, swap, uptime and processes while studying Operating Systems.
Faculty and Evaluators	Provides a practical demonstration of Linux resource management concepts in a classroom or laboratory.
Software Developers	Allows quick observation of the effect of a running program on CPU and memory usage.
Beginners in Linux	Provides a graphical alternative to remembering several command-line tools.
System Administrators	Gives a quick overview of the machine's current resource condition and the heaviest processes.

CHAPTER 2

OPERATING SYSTEM CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

An operating system acts as an intermediary between the user, the application programs and the computer hardware. Its major functions are process management, CPU scheduling, memory management, file system management, input/output management and protection. This project does not modify any of these functions; instead, it **observes** their results by reading the information that the Linux kernel makes available. The Operating Systems concepts that are related to the project are listed in Table 2.1.

Table 2.1: Operating Systems Concepts Related to the Project

Concept	Relevance to the Project
CPU management and utilisation	CPU time counters in /proc/stat are used to calculate the CPU usage percentage.
Memory management	Total, available and used physical memory are obtained from /proc/meminfo and shown as memory usage.
Virtual memory and swap	Swap total and swap free values from /proc/meminfo are used to calculate swap usage.
Process management	Process information obtained through ps is used to count processes and to find the top CPU and memory consumers.
File systems	Disk usage is obtained using df; the /proc virtual filesystem is used as the main source of kernel information.
Kernel-user interface	The kernel exposes its internal data to user programs through files in /proc and through interfaces such as sysconf().
Time management	System uptime is read from /proc/uptime and the tool refreshes information periodically using a timer.
Resource monitoring	Periodic sampling, calculation of percentages and threshold-based health indication.

2.2 CPU Management and CPU Utilization

The CPU is the most important resource of a computer because every instruction of every program is executed by it. In a multitasking system, the operating system scheduler allocates the CPU to ready processes for short periods of time. Because the switching is very fast, many programs appear to run simultaneously.

CPU utilisation is the fraction of time during which the CPU is busy executing work instead of being idle. A high utilisation indicates that the processor is heavily loaded, while a very low utilisation indicates that it is mostly idle.

In Linux, the file `/proc/stat` provides cumulative CPU time counters since system start-up. The first line, which begins with the word `cpu`, gives the combined values for all processors. The values are measured in clock ticks (jiffies) and represent the time spent in different modes, such as:

- **user** – time spent executing normal user-space processes,
- **nice** – time spent executing user-space processes with a modified (nice) priority,
- **system** – time spent executing kernel code,
- **idle** – time during which the CPU had nothing to execute,
- **iowait** – time spent waiting for input/output to complete,
- **irq** and **softirq** – time spent servicing hardware and software interrupts,
- **steal**, **guest** and **guest_nice** – time related to virtualised environments.

Since these counters only increase, a single reading does not describe the present load. The usage must be calculated from the **difference between two readings** taken at different times:

$$\text{CPU Usage (\%)} = ((\Delta\text{Total} - \Delta\text{Idle}) / \Delta\text{Total}) \times 100$$

Here, ΔTotal is the increase in the sum of all CPU time counters between the two readings, and ΔIdle is the increase in idle time. This is the principle used by the tool, and it explains why the application stores the previous reading and compares it with the next one at every refresh.

2.3 Memory Management

Main memory (RAM) holds the programs and data that are currently in use. The operating system allocates memory to processes, keeps track of which parts are in use, and reclaims memory when it is no longer needed. Linux uses **virtual memory**, in which each process sees its own address space while the kernel maps these addresses to physical memory and, when necessary, to a swap area on disk.

Swap space is an area of secondary storage used as an extension of RAM. When physical memory becomes scarce, the kernel can move less-used pages from RAM to swap. Because disk access is much slower than RAM access, heavy use of swap usually indicates memory pressure and results in reduced performance.

The file `/proc/meminfo` reports the kernel's view of memory in kilobytes. The fields that are important for this project are:

Table 2.2: Important Fields of `/proc/meminfo`

Field	Meaning
MemTotal	Total usable physical memory (RAM).
MemAvailable	Estimate of the memory available for starting new applications without swapping; it includes memory that can be reclaimed, such as cache.
SwapTotal	Total size of the swap space.
SwapFree	Amount of swap space that is currently unused.

The free-memory value alone can be misleading in Linux, because the kernel uses otherwise unused memory for caching files, and this cache can be released when programs need it. For this reason, the tool calculates used memory with the **MemAvailable** field, which gives a more realistic picture of memory pressure.

2.4 Process Management

A **process** is a program in execution. Each process has a unique process identifier (PID), its own address space, and information such as its state, CPU usage and memory usage. The kernel keeps this information in a data structure called the process control block; in Linux this structure is called `task_struct`. A process can be in states such as running, sleeping (waiting), stopped or zombie.

The operating system continuously creates, schedules and terminates processes. At any moment a typical Linux system has many processes, most of which are sleeping and waiting for an event. A process that uses a large share of CPU or memory can slow down the entire system, so identifying such processes is one of the main purposes of resource monitoring.

Linux exposes process information as directories named by the PID under `/proc`. The `ps` utility reads this information and prints it in a readable table with columns such as PID, command name, CPU percentage and memory percentage. The Linux Resource Monitoring Tool uses `ps` to obtain this data, counts the processes, and finds the processes with the highest CPU and memory percentages.

Note: In the standard `ps` utility, the %CPU column is calculated as the CPU time used by a process divided by the time the process has been running. It therefore represents the average usage over

the lifetime of the process and not necessarily the instantaneous usage. The %MEM column is the percentage of physical memory occupied by the process.

2.5 File Systems and Linux /proc

A file system organises data on storage devices into files and directories and provides operations to create, read, write and delete them. Linux follows the principle that many system resources are represented as files, so that the same simple file operations can be used to access them.

The **/proc filesystem** (procfs) is a **virtual filesystem**. Its files do not exist on the disk; they are generated by the kernel in memory at the time they are read. It acts as an interface between user programs and kernel data structures. A program can therefore obtain kernel information by opening and reading files in /proc using ordinary file input, without any special library.

Table 2.3: /proc Files Used in the Project

File	Information Provided	Used For
/proc/stat	Cumulative CPU time counters since boot	CPU usage and CPU usage history
/proc/meminfo	Memory and swap statistics	Memory usage, swap usage, total memory and memory usage history
/proc/uptime	Time since the system was started	System uptime

Disk usage, in contrast, is not obtained from /proc. It is obtained from the `df` utility, which reports how much space is used and available on mounted file systems.

2.6 Resource Monitoring

Resource monitoring is the process of observing the usage of system resources over time. The commonly monitored resources are CPU, memory, disk, network and processes. A monitoring tool normally follows the following steps: it **collects** raw data from the system, **processes** the data to produce meaningful values, **presents** the values to the user, and **repeats** the cycle at regular intervals (periodic sampling or polling).

Two ideas are important in monitoring. The first is **utilisation**, which tells how much of a resource is in use. The second is the **trend**, which tells how the usage changes with time. A single value may be misleading, whereas a history graph clearly shows sudden increases and long-term behaviour. For this reason, the project provides history graphs for CPU and memory usage.

Monitoring itself consumes resources. A good monitoring tool should therefore be lightweight and should not refresh too frequently. The tool developed in this project reads small files from `/proc` and runs two small utilities at a fixed refresh interval.

2.7 Linux Commands and Utilities

Linux provides many command-line utilities for observing the system, for example `top`, `htop`, `free`, `uptime`, `ps` and `df`. Some of these utilities can also be used to verify the values displayed by the tool. The project uses the interfaces shown in Table 2.4, which distinguishes between the `/proc` filesystem, Linux utilities, and programming interfaces.

Table 2.4: Interfaces and Utilities Used by the Project

Category	Name	Purpose in the Project
Virtual filesystem	<code>/proc/stat</code> <code>/proc/meminfo</code> <code>/proc/uptime</code>	CPU counters Memory and swap information System uptime
Linux utility (command)	<code>ps</code>	Process list with PID, name, CPU and memory percentage
Linux utility (command)	<code>df</code>	Disk usage
System interface (library function)	<code>sysconf()</code>	Number of CPU cores
C library functions	<code>popen()</code> , <code>pclose()</code>	Run a command (<code>ps</code> , <code>df</code>) and read its output; then close it
GTK4 / GLib	GTK4 widgets and timer	Dashboard, buttons, dialog and periodic refresh
Cairo / GTK drawing	Drawing area and Cairo drawing functions	CPU and Memory history graphs

`popen()` and `pclose()` are C library functions that start a command through the shell and provide a stream for reading its output. Internally, the C library creates a child process and a pipe for this purpose, but this happens inside the library. The source code of the project does not call `fork()`, `exec()`, `wait()`, `pipe()` or thread-creation functions directly, and it does not implement inter-process communication, semaphores or shared memory by itself.

2.8 Proposed Approach

The proposed approach is to build a single GTK4 desktop application in C++ that periodically collects resource information from the Linux system, calculates usage values, and displays them on a dashboard. The approach consists of the following stages:

1. **Data collection:** Read `/proc/stat`, `/proc/meminfo` and `/proc/uptime`; run `df` and `ps`; and obtain the number of CPU cores using `sysconf()`.
2. **Processing:** Convert the collected raw values into usage percentages, uptime in readable form, process count, top CPU consumer, top memory consumer and system health status.
3. **History maintenance:** Store recent CPU and memory usage values so that history graphs can be drawn.
4. **Presentation:** Display all values in a card-based GTK4 dashboard along with two history graphs.
5. **Refreshing:** Repeat the above stages automatically using a timer, and also on request using the Refresh button.

2.9 Advantages of the Proposed Approach

- Uses the real data supplied by the Linux kernel, so the values are genuine and not simulated.
- No external monitoring library is needed; only the standard C++ facilities, GTK4 and standard Linux utilities are used.
- A graphical dashboard is easier to read than raw text from several commands.
- History graphs make changes in CPU and memory usage visible over time.
- Automatic refresh keeps the information current without user effort.
- The simple health indication helps a beginner to interpret the values.
- The modular design makes it easy to add further monitors in the future.
- Runs on Ubuntu/Linux, including Ubuntu under WSL, which is convenient for students.

CHAPTER 3

SYSTEM REQUIREMENTS

This chapter lists the hardware and software required to develop, compile and run the Linux Resource Monitoring Tool. The project does not require any special or costly hardware; an ordinary computer or laptop that can run Ubuntu (natively or through WSL) is sufficient.

3.1 Hardware Requirements

Table 3.1: Hardware Requirements

Component	Requirement
Computer	Desktop computer or laptop
Processor	Dual-core processor or better
RAM	Minimum 4 GB
Storage	Sufficient free disk space for Ubuntu/WSL, the development tools and the project files
Display	Standard display capable of showing a graphical desktop application
Special hardware	Not required

3.2 Software Requirements

Table 3.2: Software Requirements

Software	Purpose
Windows with WSL, or native Ubuntu/Linux	Operating system environment for development and execution
Ubuntu	Linux distribution used for development and testing
GCC / g++	C++ compiler used to build the program
GTK4 (development package)	Graphical user interface toolkit
pkg-config	Provides the compiler and linker options required for GTK4
VS Code	Source code editor
Linux utilities (ps, df)	Process and disk information

3.3 Operating System Environment – Ubuntu / WSL

The project is developed and tested in **Ubuntu running through the Windows Subsystem for Linux (WSL)**. WSL allows a Linux distribution such as Ubuntu to run on a Windows computer without using a separate virtual machine or dual-boot installation. It provides a Linux command-line environment, the Linux directory structure, the /proc filesystem, and standard Linux utilities, which are all required by this project.

Because the tool has a graphical interface, the Ubuntu environment must be able to display Linux GUI applications. On supported versions of Windows this is provided by WSL's graphical application support. The same program can also be compiled and executed on a native Ubuntu/Linux installation without any change to the source code.

Note: Values such as total memory, CPU cores and uptime displayed when the tool runs under WSL describe the Linux environment seen by Ubuntu in WSL, which may differ from the values of the Windows host.

3.4 Development Tools and Compiler

The program is written in C++ and compiled using the GNU Compiler Collection (g++). The GTK4 development files are located using pkg-config. VS Code is used as the editor. On Ubuntu, the required packages can be installed using the apt package manager (see Chapter 7). The development tools used are summarised below.

Table 3.3: Development Tools

Tool	Role
g++ (GCC)	Compiles the C++ source file into an executable
pkg-config	Supplies GTK4 include paths and libraries to the compiler
Terminal (Ubuntu/WSL)	Used for compilation, execution and checking command outputs
VS Code	Editing, organising and viewing the source code

CHAPTER 4

TECHNOLOGY AND TOOLS USED

This chapter describes the programming language, libraries, interfaces and tools used in the project and the reason for choosing each of them.

4.1 C++ Programming

C++ is a general-purpose, compiled programming language that supports both low-level system programming and high-level abstractions. It is suitable for this project because it can easily read files, parse text, perform calculations, call C library functions such as `sysconf()` and `popen()`, and work directly with the GTK4 library, which is written in C. C++ also provides standard containers and string-handling facilities that simplify the storage and processing of readings and history values.

4.2 GTK4

GTK (the GIMP Toolkit) is a widely used open-source toolkit for creating graphical user interfaces on Linux. GTK4 is its current major version. It provides windows, containers, labels, buttons, dialogs and drawing areas, and it handles user events through a signal mechanism. It also provides timer functions through GLib, which are used in this project for periodic refreshing. In this project, GTK4 is used to build the dashboard window, the cards that show resource values, the Refresh and About Project buttons, the About Project dialog, and the areas in which the history graphs are drawn.

4.3 GCC / g++ Compiler

GCC is the GNU Compiler Collection, and g++ is its C++ front end. It translates the C++ source code into an executable program for Linux. The compilation command used in this project is:

```
g++ main.cpp -o monitor $(pkg-config --cflags --libs gtk4)
```

The part `$(pkg-config --cflags --libs gtk4)` is replaced by the compiler options and libraries required by GTK4.

4.4 Linux /proc Virtual File System

The /proc filesystem is the main source of system information in the project. The files /proc/stat, /proc/meminfo and /proc/uptime are read using file input operations in C++. Since these files are generated by the kernel, they contain up-to-date values every time they are read and they do not occupy disk space. Their purpose in the project is summarised in Table 2.3.

4.5 Linux Commands and Utilities

Two Linux utilities are used by the project. The `ps` utility provides the list of processes together with their names, PIDs, CPU percentage and memory percentage. The `df` utility provides disk usage. The program runs these utilities with the C library functions `popen()` and reads their text output, then closes the stream with `pclose()`. In addition, the C library function `sysconf()` is used to obtain the number of CPU cores.

Clarification: /proc files, Linux utilities (ps, df) and library functions (sysconf, popen, pclose) are three different kinds of sources. The /proc files are read directly by the program, the utilities are executed as commands, and the library functions are called from the C++ code.

4.6 Cairo / GTK Drawing

Cairo is a two-dimensional vector graphics library that is used by GTK for drawing. GTK4 provides a drawing area widget in which a program can draw using Cairo functions such as drawing lines, rectangles and text. In this project, this functionality is used to draw the CPU Usage History and Memory Usage History graphs from the stored usage values.

4.7 VS Code

Visual Studio Code is a lightweight source code editor. It provides syntax highlighting, code navigation and an integrated terminal, and it can work with WSL, so the code can be edited and compiled in the Ubuntu environment from the same window.

4.8 WSL / Ubuntu

Ubuntu is a popular Linux distribution that provides the `apt` package manager and the GCC and GTK4 packages. WSL makes Ubuntu available inside Windows. Together they provide the Linux environment, the /proc filesystem and the utilities needed to develop and test the project on a normal student laptop.

Table 4.1: Summary of Technologies Used

Technology	Category	Use in the Project
C++	Programming language	Implementation of all logic
GTK4	GUI toolkit	Dashboard, buttons, dialog, timer
Cairo / GTK drawing	Graphics	History graphs
/proc	Virtual filesystem	CPU, memory, swap and uptime data
ps, df	Linux utilities	Process and disk information
sysconf(), popen(), pclose()	C library functions	CPU cores, command execution
g++, pkg-config	Build tools	Compilation
VS Code, Ubuntu/WSL	Development environment	Editing, testing and execution

CHAPTER 5

SYSTEM ARCHITECTURE AND DESIGN

This chapter explains the overall design of the Linux Resource Monitoring Tool, including its architecture, workflow, data flow, modules, input/output design and the design of the graphical interface.

5.1 Overall System Architecture

The tool follows a layered architecture. The lowest layer is the Linux system itself, which supplies the raw data. Above it are the data collection layer, the resource processing layer, and finally the GTK4 user interface that shows the monitoring dashboard. The architecture is shown in Figure 5.1.

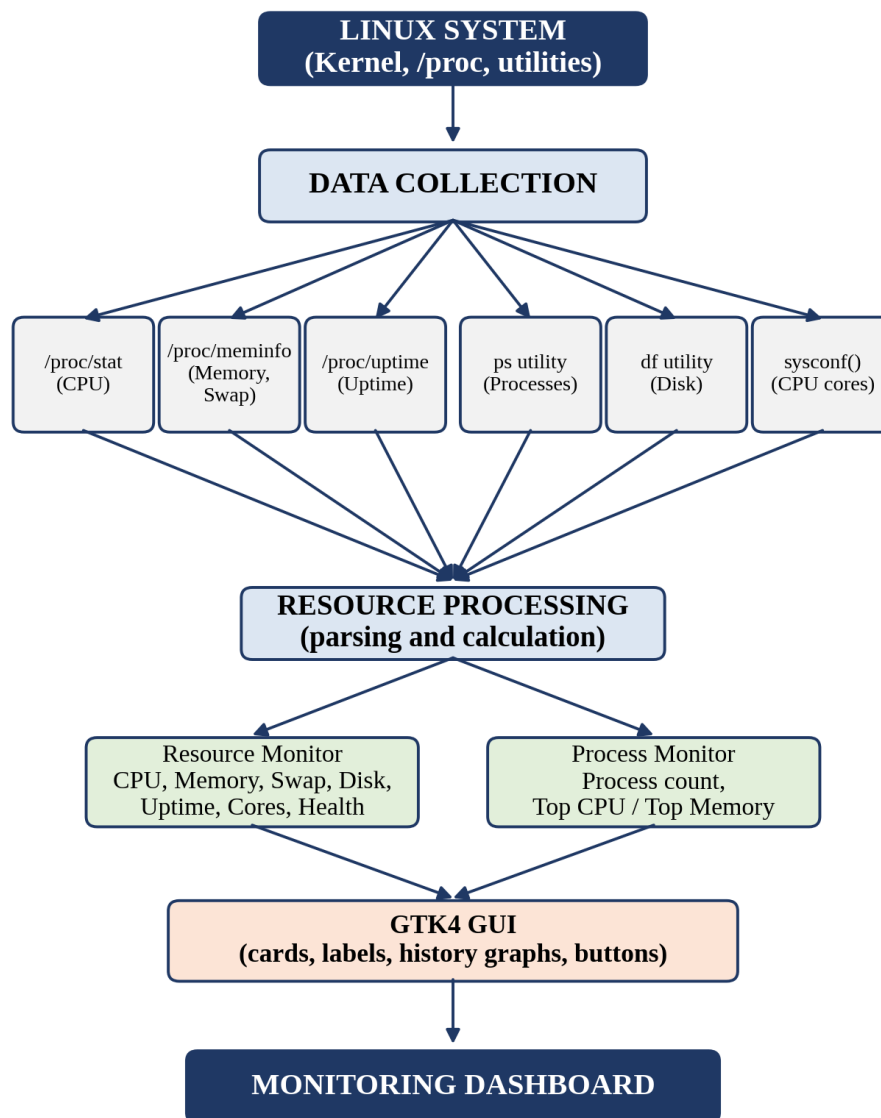


Figure 5.1: Overall System Architecture

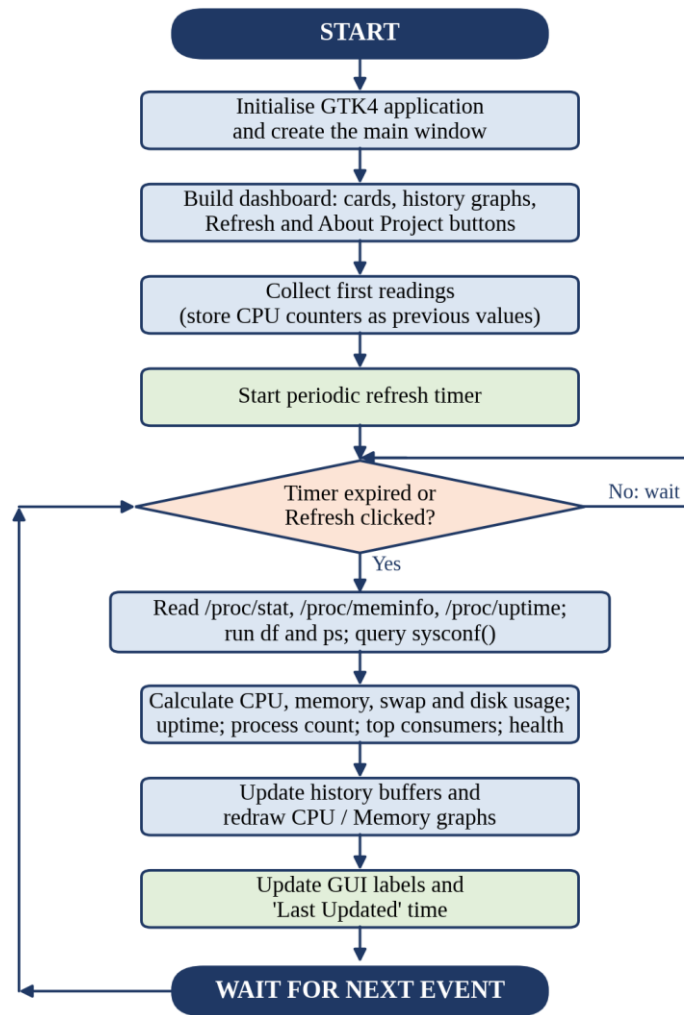
The components of the architecture are explained below.

Table 5.1: Components of the System Architecture

Component	Description
Linux System	The operating system kernel, the /proc virtual filesystem and the Linux utilities from which all information is obtained.
Data Collection	The part of the program that reads /proc/stat, /proc/meminfo and /proc/uptime, executes df and ps through popen(), and calls sysconf().
Resource Processing	Parses the raw text and numbers and calculates usage percentages, uptime, process count, top consumers and system health.
Resource Monitor	Logical group of functions handling CPU, memory, swap, disk, uptime, total memory, CPU cores and health.
Process Monitor	Logical group of functions handling the process count and identification of the top CPU and top memory consumers. It is a logical module and not a separate window.
GTK4 GUI	Displays the values in cards, draws the history graphs and provides the Refresh and About Project buttons.
Monitoring Dashboard	The final output seen by the user.

5.2 System Workflow

When the application is started, it initialises GTK4 and creates the main window with all dashboard elements. The first readings are collected, the refresh timer is started and the application waits for events. Whenever the timer expires or the user clicks the Refresh button, all information is read again, the values are calculated, the history is updated, the graphs are redrawn and the labels are updated. The complete workflow is shown in Figure 5.2.

**Figure 5.2: System Workflow**

5.3 Data Flow

Data flows from the data sources to the user in four stages: data sources, collection, processing and presentation. The `/proc` files, the output of `ps` and `df`, and the value returned by `sysconf()` are collected by the C++ program. The processing stage converts them into resource values and process information. Finally, the values are shown on the GTK4 widgets, and the CPU and memory values are also stored in history buffers for the graphs.

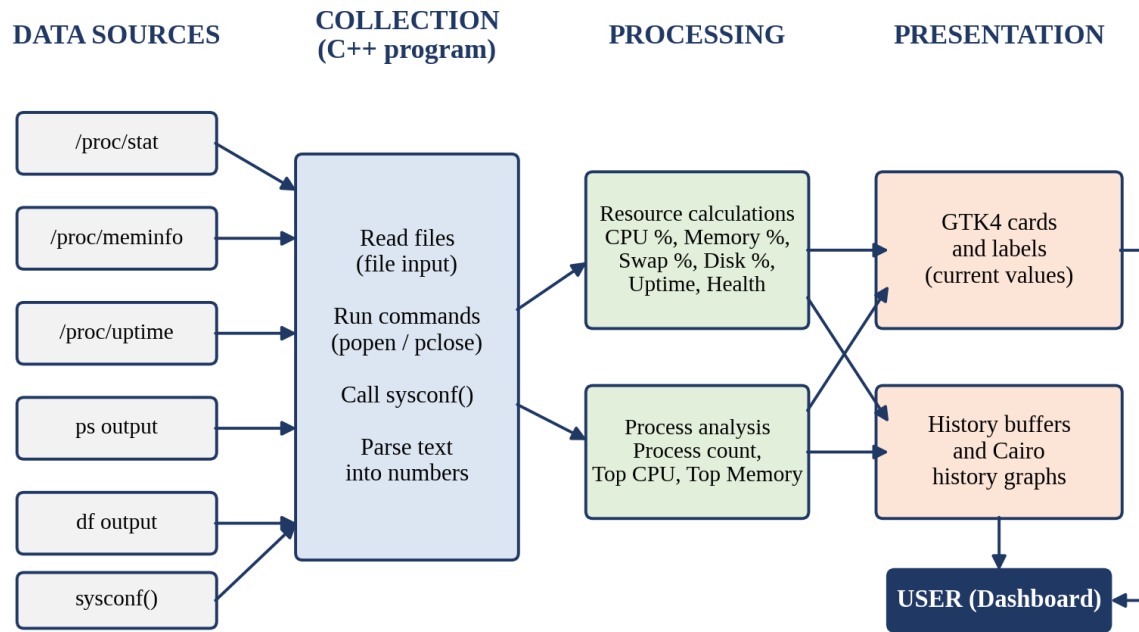


Figure 5.3: Data Flow Diagram

5.4 Module Description

The application is divided into logical modules. The modules are functional divisions of the program and are described in Table 5.2.

Table 5.2: Module Description

Module	Function	Data Source
System Information Module	Provides total memory, number of CPU cores, uptime and running process count	/proc/meminfo, sysconf(), /proc/uptime, ps
CPU Monitoring Module	Calculates CPU usage by comparing two readings	/proc/stat
Memory Monitoring Module	Calculates used memory and memory usage percentage	/proc/meminfo
Swap Monitoring Module	Calculates swap usage percentage	/proc/meminfo
Disk Monitoring Module	Obtains disk usage percentage	df utility
Process Monitoring Module	Finds the top CPU consumer and top memory consumer	ps utility
System Health Module	Classifies resource usage as Normal, Warning or High	Calculated usage values
History Graph Module	Stores recent CPU and memory values and draws the graphs	Calculated usage values
GUI Module	Builds the window, cards and buttons using GTK4	All calculated values
Refresh Module	Updates all values periodically (timer) and on button click	GTK4 timer, Refresh button
About Project Module	Shows information about the project in a dialog	Static project information

5.5 Input and Output Design

The application does not require the user to type any input. Its inputs are the system data read from the Linux system and the user's mouse actions on the buttons. The outputs are the values and graphs shown on the dashboard.

Table 5.3: Input and Output Design

Type	Item	Description
Input	/proc/stat, /proc/meminfo, /proc/uptime	Kernel-provided CPU, memory, swap and uptime data
Input	Output of ps and df	Process and disk information
Input	Return value of sysconf()	Number of CPU cores
Input	Timer event	Triggers an automatic refresh
Input	Button clicks	Refresh and About Project
Output	Resource cards	CPU, memory, disk, swap, total memory and CPU cores
Output	History graphs	CPU Usage History and Memory Usage History
Output	Process and system information	Top CPU consumer, top memory consumer, uptime, running processes, system health, last updated time
Output	About dialog	Information about the project

5.6 GUI Design

The graphical interface is a single main dashboard window. It uses a card-based layout in which each item of information is displayed in its own area with a heading and a value, so that the user can read the information quickly. The history graphs are placed in the dashboard together with the other information. The main window contains the elements listed in Table 5.4.

Table 5.4: Elements of the Main Dashboard

No.	Element	Description
1	CPU Usage Card	Shows the current CPU usage percentage.
2	Memory Usage Card	Shows the percentage of physical memory in use.
3	Disk Usage Card	Shows the disk usage obtained from df.
4	Swap Usage Card	Shows the percentage of swap space in use.
5	Total Memory Card	Shows the total physical memory of the system.
6	CPU Cores Card	Shows the number of CPU cores.
7	CPU Usage History	A graph showing recent CPU usage values.
8	Memory Usage History	A graph showing recent memory usage values.
9	Top CPU Consumer	Shows the process with the highest CPU usage.
10	Top Memory Consumer	Shows the process with the highest memory usage.

No.	Element	Description
11	System Uptime	Shows the time for which the system has been running.
12	Running Processes	Shows the number of processes.
13	System Health	Shows Normal, Warning or High according to usage thresholds.
14	Last Updated	Shows the time at which the information was last refreshed.
15	Refresh Button	Updates all information immediately.
16	About Project Button	Opens the About Project dialog.

The design aims at simplicity: all information is visible in a single window without any menu, and the only user actions are pressing the Refresh button and the About Project button. The final dashboard does not contain a separate detailed Process Monitor window; process information is limited to the process count and the top CPU and memory consumers.

CHAPTER 6

IMPLEMENTATION

This chapter explains how each feature of the Linux Resource Monitoring Tool is implemented. The explanation is given at the level of methods and algorithms. Worked examples use illustrative numbers to show how a calculation is performed; they are not measurements taken from the tool.

6.1 Module-wise Implementation

The application is implemented in C++ as a GTK4 program compiled from a single source file, `main.cpp`. Functionally, it can be divided into data collection, calculation, history maintenance, and user interface parts. Table 6.1 summarises how each feature obtains its data and which Linux mechanism it relies on.

Table 6.1: Feature-wise Implementation Summary

Feature	Data Source / Mechanism	Type of Source
CPU Usage	/proc/stat, two readings compared	Virtual filesystem
Memory Usage	MemTotal and MemAvailable from /proc/meminfo	Virtual filesystem
Swap Usage	SwapTotal and SwapFree from /proc/meminfo	Virtual filesystem
Disk Usage	df command executed using <code>popen()</code>	Linux utility
Total Memory	MemTotal from /proc/meminfo	Virtual filesystem
CPU Cores	<code>sysconf()</code>	System interface (library function)
System Uptime	/proc/uptime	Virtual filesystem
Running Processes	ps command executed using <code>popen()</code>	Linux utility
Top CPU / Memory Consumer	ps output (PID, name, CPU %, memory %)	Linux utility
System Health	Threshold comparison of usage values	Program logic
History Graphs	Stored usage values drawn with Cairo	GTK4 / Cairo drawing
Automatic Refresh	GTK4 / GLib timer	GUI toolkit

The overall execution of the program follows the event-driven model of GTK4. The program creates the application and the window, builds the interface, and then hands control to the GTK

main loop. From that point onwards, the program reacts to events: a timer event performs the automatic update, a button-click event performs the manual update or opens the About dialog, and a draw event paints the history graphs.

The main dashboard produced by the program is shown in Figure 6.1, and the history graphs it contains are shown in Figure 6.2.



Figure 6.1: Linux Resource Monitoring Tool – Main Dashboard

6.2 CPU Monitoring Implementation

CPU usage is calculated from the aggregate `cpu` line of `/proc/stat`. The program reads the numeric fields of this line, adds them to obtain the total CPU time, and takes the idle time. Because the counters are cumulative since boot, the program stores the first reading as the previous reading. At the next update, it reads the counters again, subtracts the previous values to obtain the change during the interval, and calculates the percentage of non-idle time.

Algorithm 6.1: CPU Usage Calculation

1. Read the first line ("cpu") of `/proc/stat`.
2. Store the total time and the idle time as the previous values.
3. After the refresh interval, read `/proc/stat` again.
4. Calculate the current total time and idle time.
5. $\text{deltaTotal} = \text{currentTotal} - \text{previousTotal}$
 $\text{deltaIdle} = \text{currentIdle} - \text{previousIdle}$
6. $\text{cpuUsage} = ((\text{deltaTotal} - \text{deltaIdle}) / \text{deltaTotal}) * 100$
7. Replace the previous values with the current values.
8. Display `cpuUsage` and add it to the CPU history.
9. Repeat periodically.

Worked example (illustrative): Suppose the first reading gives a total of 1,000,000 ticks with 900,000 idle ticks, and the second reading gives a total of 1,000,400 ticks with 900,300 idle ticks. Then $\Delta\text{Total} = 400$ and $\Delta\text{Idle} = 300$, so $\text{CPU usage} = ((400 - 300) / 400) \times 100 = 25\%$.

The calculation must avoid division by zero when the total does not change between two readings; in such a case the previous displayed value can be retained.

6.3 Memory Monitoring Implementation

Memory usage is calculated from `/proc/meminfo`. The program reads the `MemTotal` and `MemAvailable` fields (in kilobytes). The used memory is the difference between the two, and the usage percentage is the used memory divided by the total memory.

Algorithm 6.2: Memory Usage Calculation

```
1. Open /proc/meminfo.
2. Read MemTotal.
3. Read MemAvailable.
4. usedMemory = MemTotal - MemAvailable
5. memoryUsage = (usedMemory / MemTotal) * 100
6. Display memoryUsage and add it to the memory history.
```

$$\text{Memory Usage (\%)} = ((\text{MemTotal} - \text{MemAvailable}) / \text{MemTotal}) \times 100$$

Worked example (illustrative): If `MemTotal` is 8,000,000 kB and `MemAvailable` is 5,600,000 kB, the used memory is 2,400,000 kB and the memory usage is $(2,400,000 / 8,000,000) \times 100 = 30\%$.

6.4 Disk Monitoring Implementation

Disk usage is not available in `/proc`, so the project uses the Linux `df` utility. The program starts the command using the C library function `popen()`, reads the text printed by `df`, extracts the usage percentage from the output, and closes the stream using `pclose()`. The extracted value is displayed on the Disk Usage card.

Algorithm 6.3: Disk Usage Retrieval

```
1. Execute the df command using popen().
2. Read the output line(s) produced by df.
3. Extract the used-space percentage from the output.
4. Close the stream using pclose().
5. Convert the extracted text to a number.
6. Display the disk usage on the Disk Usage card.
```

The `df` command itself obtains the information from the file system through the operating system; the project only runs the command and reads its result.

6.5 Swap Monitoring Implementation

Swap usage is calculated from the `SwapTotal` and `SwapFree` fields of `/proc/meminfo`. If the system has no swap space (`SwapTotal` is zero), the usage is taken as zero to avoid division by zero.

Algorithm 6.4: Swap Usage Calculation

1. Read `SwapTotal` and `SwapFree` from `/proc/meminfo`.
2. If `SwapTotal` is 0, set `swapUsage` = 0.
3. Otherwise, `swapUsed` = `SwapTotal` - `SwapFree`
4. `swapUsage` = (`swapUsed` / `SwapTotal`) * 100
5. Display `swapUsage` on the Swap Usage card.

$$\text{Swap Usage (\%)} = ((\text{SwapTotal} - \text{SwapFree}) / \text{SwapTotal}) \times 100$$

6.6 Total Memory and CPU Core Information

Total memory: The `MemTotal` value read from `/proc/meminfo` is in kilobytes. It is converted into a more readable unit, such as megabytes or gigabytes, and shown on the Total Memory card.

CPU cores: The number of processor cores available to the system is obtained using the system interface `sysconf()`, which returns configuration information from the operating system. The returned number is shown on the CPU Cores card. This value does not change during execution; it describes the Linux environment (including a WSL environment) in which the program runs.

6.7 System Uptime

The file `/proc/uptime` contains two numbers. The first is the total time in seconds for which the system has been running. The program reads the first number and converts it into a readable form.

Algorithm 6.5: System Uptime Calculation

1. Read the first number (seconds) from `/proc/uptime`.
2. `days` = `seconds` / 86400
3. `hours` = (`seconds` % 86400) / 3600
4. `minutes` = (`seconds` % 3600) / 60
5. Display the uptime in days, hours and minutes.

Example (illustrative): An uptime value of 93,784 seconds corresponds to 1 day, 2 hours and 3 minutes.

6.8 Process Monitoring

Process information is obtained using the Linux `ps` utility. The program runs `ps` through `popen()` and reads its output, in which each line represents one process and contains the process name, PID, CPU percentage and memory percentage. The program reads the lines one by one and stores the values. The number of lines read gives the number of processes, which is displayed as the **Running Processes** value.

Algorithm 6.6: Process Monitoring

1. Execute `ps` using `popen()` to obtain process information.
2. Read each output line: process name, PID, CPU %, memory %.
3. Skip the monitoring application itself where appropriate.
4. Count the processes read.
5. Compare the CPU % values and record the highest one.
6. Compare the memory % values and record the highest one.
7. Close the stream using `pclose()`.
8. Display the process count and the two top consumers.

The monitoring application itself is excluded from the comparison where appropriate, so that the tool does not report itself as the heaviest process merely because it is refreshing the display.

6.9 Top CPU Consumer

While reading the `ps` output, the program keeps the highest CPU percentage seen so far together with the name and PID of that process. When all lines have been read, the stored process is the top CPU consumer. Its name and CPU percentage are displayed on the **Top CPU Consumer** area of the dashboard.

Algorithm 6.7: Top CPU Consumer

1. Set `maxCpu = 0` and `topCpuProcess = none`.
2. For each process (excluding this application):
 if `process.cpu > maxCpu` then
 `maxCpu = process.cpu`
 `topCpuProcess = process (name, PID)`
3. Display `topCpuProcess` and `maxCpu`.

Note: In the standard `ps` utility, the CPU percentage reflects the CPU time used by a process relative to the time it has been running. It is therefore a lifetime average, and the process shown may differ from the one shown by tools such as `top`, which report usage over a short recent interval.

6.10 Top Memory Consumer

In the same way, the program keeps the highest memory percentage found in the `ps` output together with the name and PID of the process. The memory percentage reported by `ps` is the share of physical memory occupied by the process. The result is displayed in the **Top Memory Consumer** area of the dashboard.

Algorithm 6.8: Top Memory Consumer

```
1. Set maxMem = 0 and topMemProcess = none.
2. For each process (excluding this application):
    if process.mem > maxMem then
        maxMem = process.mem
        topMemProcess = process (name, PID)
3. Display topMemProcess and maxMem.
```

6.11 System Health Calculation

The System Health indicator provides a simple interpretation of the current resource condition. The usage percentage is compared with two fixed threshold values, 70% and 90%, and the status is shown as Normal, Warning or High.

Table 6.2: System Health Thresholds

Resource Usage	Status	Interpretation
Below 70%	Normal	Resources are used comfortably.
70% to 90%	Warning	Usage is considerable; the user should observe the system.
Above 90%	High	Resources are heavily used and performance may be affected.

Algorithm 6.9: System Health Evaluation

```
1. Take the current usage percentage.
2. If usage < 70 then status = "Normal"
3. Else if usage <= 90 then status = "Warning"
4. Else status = "High"
5. Display the status in the System Health area.
```

This is a **rule-based** indication that uses fixed thresholds. It is not an advanced or AI-based prediction system, and it does not forecast future behaviour; it only gives a quick interpretation of the present readings.

6.12 CPU and Memory History Graphs

The dashboard contains two history graphs: **CPU Usage History** and **Memory Usage History**. At every update, the newly calculated CPU and memory usage percentages are added to two history lists that keep a limited number of the most recent values. When the lists are full, the oldest value is discarded. Each graph is drawn in a drawing area widget using Cairo drawing functions, and the line connects the stored values from the oldest to the newest, so that the user can see how the usage has changed over the recent period.

Algorithm 6.10: History Graph Update

1. After each update, append the new usage value to the history list.
2. If the list has more than the allowed number of values, remove the oldest value.
3. Request a redraw of the drawing area.
4. In the draw function:
 - a. Draw the graph background and axes.
 - b. Map each stored value (0-100%) to a vertical position.
 - c. Join the points with lines using Cairo.



Figure 6.2: CPU and Memory Usage History

6.13 GTK4 GUI Implementation

The user interface is created using GTK4 in the C++ program. The main steps are:

1. Create the GTK application and connect the activation handler.
2. In the handler, create the main application window and set its title and size.
3. Create container widgets to arrange the dashboard in rows and columns.
4. Create the cards: each card uses labels for the heading and for the value.
5. Create the drawing areas for the CPU and memory history graphs.
6. Create the labels for the top consumers, uptime, running processes, system health and last updated time.

7. Create the Refresh and About Project buttons and connect them to their handlers.
8. Show the window and start the GTK main loop, which waits for events.

GTK4 programs are event-driven. Instead of running in a continuous loop written by the programmer, the program registers functions (callbacks) for events such as timer expiry, button clicks and drawing, and the GTK main loop calls them when the events occur. This model keeps the interface responsive.

6.14 Automatic Refresh

Automatic refresh is implemented with a GTK4/GLib timer. A timer is registered at start-up with a fixed interval and a callback function. When the interval elapses, the GTK main loop calls the callback. The callback performs a complete update and returns a value that tells the timer to continue, so the update repeats for as long as the application is running.

Algorithm 6.11: Automatic Refresh

1. At start-up, register a timer with a fixed interval.
2. When the timer fires, the callback is executed.
3. In the callback:
 - a. Read CPU, memory, swap, uptime, disk and process data.
 - b. Calculate all values and the system health.
 - c. Update the history lists and redraw the graphs.
 - d. Update all labels and the Last Updated time.
4. Return from the callback so that the timer continues.

The **Last Updated** value shows the system clock time at which the most recent update was performed, which lets the user confirm that the data is current.

6.15 Refresh Button

The Refresh button allows the user to update the dashboard immediately without waiting for the next timer event. The button is connected to a handler function through the GTK signal mechanism. When the button is clicked, the handler obtains the latest values and updates the dashboard, including the Last Updated time.

6.16 About Project Module

The About Project button opens a dialog that displays information about the project. The dialog is created with GTK4 widgets and is shown on top of the main window. It does not collect any system data. Its purpose is to identify the project and give the user basic information about the application.

CHAPTER 7

EXECUTION AND DEMONSTRATION

This chapter explains how to compile and run the Linux Resource Monitoring Tool in Ubuntu/WSL, the kind of output the tool produces, where the screenshots of the application are placed, and how the tool demonstrates Operating Systems concepts.

7.1 Compilation in Ubuntu/WSL

The program is compiled in the Ubuntu terminal. If the required packages are not yet installed, they can be installed first using the apt package manager:

```
sudo apt update
sudo apt install build-essential pkg-config libgtk-4-dev
```

After the packages are installed, the source file `main.cpp` is compiled using the following command:

```
g++ main.cpp -o monitor $(pkg-config --cflags --libs gtk4)
```

The meaning of the parts of the command is given in Table 7.1.

Table 7.1: Explanation of the Compilation Command

Part	Meaning
g++	The GNU C++ compiler.
main.cpp	The source file of the project.
-o monitor	Names the output executable file as monitor.
pkg-config --cflags --libs gtk4	Supplies the include paths and libraries required for GTK4.

7.2 Execution Commands

After a successful compilation, the application is started from the same directory with:

```
./monitor
```

The application is developed and tested in Ubuntu through WSL. When it is started, the GTK4 dashboard window opens and begins to show the system values. If a window does not appear when running under WSL, the user should check that graphical application support is available

in the WSL installation. On native Ubuntu/Linux, the same command is used in a terminal running inside the desktop session.

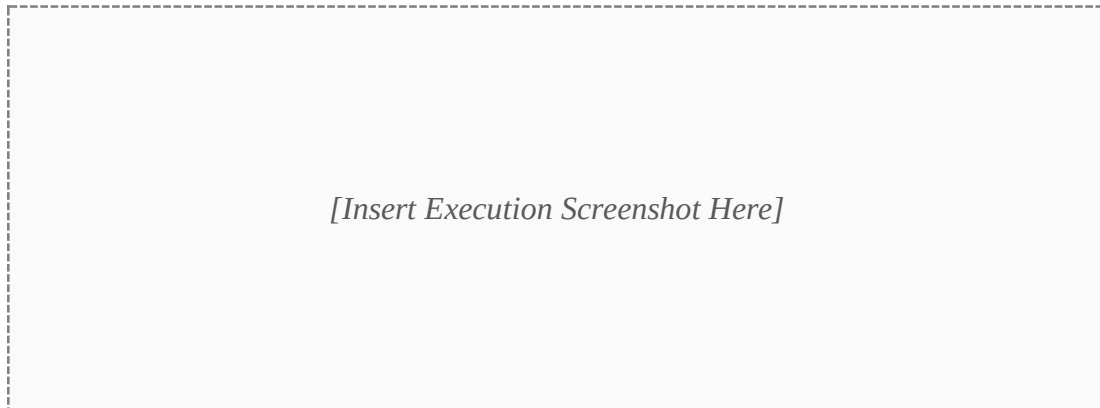


Figure 7.1: Project Execution in Ubuntu/WSL

7.3 Sample Output

The dashboard presents the values listed in Table 7.2. The values in the table are **representative examples** that show the type and format of the information displayed; the actual values depend on the system on which the tool is run and change with its load.

Table 7.2: Representative Dashboard Values (Format Illustration)

Dashboard Item	Example of Displayed Information
CPU Usage	A percentage, for example 25%
Memory Usage	A percentage, for example 30%
Disk Usage	A percentage, for example 45%
Swap Usage	A percentage, for example 0%
Total Memory	Size of RAM, for example 8 GB
CPU Cores	A number, for example 4
Top CPU Consumer	Process name with its CPU percentage
Top Memory Consumer	Process name with its memory percentage
System Uptime	Days, hours and minutes, for example 1 day 2 hours 3 minutes
Running Processes	A number of processes
System Health	Normal, Warning or High
Last Updated	Time of the latest refresh

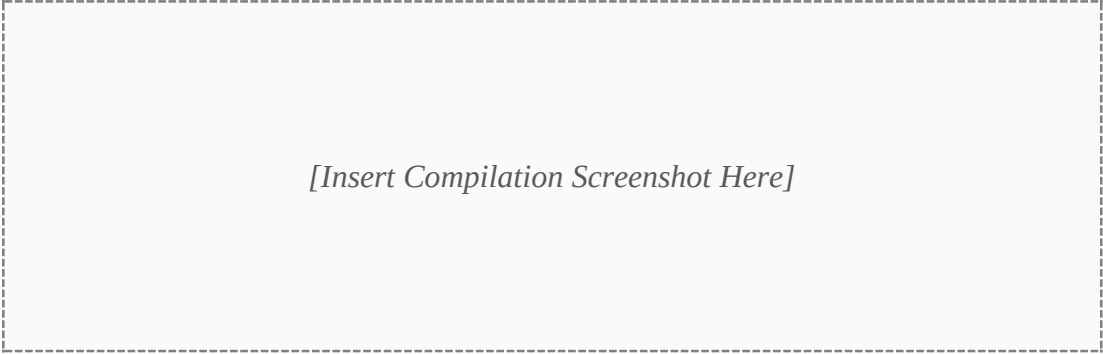
The following test scenarios describe the behaviour expected from the application and can be used to verify it. The displayed values can also be compared with the output of standard Linux commands such as `free`, `df -h`, `uptime` and `top`.

Table 7.3: Test Scenarios and Expected Behaviour

No.	Scenario	Expected Behaviour
1	Start the application with <code>./monitor</code>	The dashboard window opens and displays all cards, graphs and buttons.
2	Wait without any action	Values, graphs and the Last Updated time change automatically at each refresh.
3	Click the Refresh button	All values and the Last Updated time are updated immediately.
4	Run a CPU-intensive program in another terminal	CPU usage rises, the CPU graph shows an increase, and the program may appear as the top CPU consumer.
5	Run a memory-consuming program	Memory usage rises and the Memory graph shows an increase; the program may appear as the top memory consumer.
6	Observe usage below 70%, between 70% and 90%, and above 90%	System Health shows Normal, Warning and High respectively.
7	Click the About Project button	The About Project dialog opens with information about the project.
8	Compare with <code>free</code> , <code>df -h</code> and <code>uptime</code>	The memory, disk and uptime values are consistent with the command outputs.

7.4 GUI Screenshots

The screenshots of the final application should be placed in this section. The main dashboard and the history graphs are shown in Figure 6.1 and Figure 6.2 in Chapter 6, and the execution of the project is shown in Figure 7.1. The compilation of the project and the About Project dialog are shown in Figure 7.2 and Figure 7.3.



[Insert Compilation Screenshot Here]

Figure 7.2: Compilation of the Project in Ubuntu/WSL



[Insert About Project Dialog Screenshot Here]

Figure 7.3: About Project Dialog

7.5 Demonstration of OS Concepts

The project demonstrates several Operating Systems concepts through real data from the running Linux system. Table 7.4 shows where each concept can be observed.

Table 7.4: Operating Systems Concepts Demonstrated by the Project

OS Concept	How it is Demonstrated
CPU utilisation	CPU usage is calculated from the CPU time counters in /proc/stat and shown with a history graph.
Memory management	Used memory and total memory are derived from /proc/meminfo; the memory history graph shows changes over time.
Virtual memory / swap	Swap usage is calculated from SwapTotal and SwapFree.
Process management	The number of processes and the top CPU and memory consuming processes are obtained from ps.
File systems	The /proc virtual filesystem is used as a data source; disk usage is obtained with df.
Kernel-user interface	The kernel exposes information through /proc files and the sysconf() interface, which the program reads.
Time and periodic sampling	System uptime is read from /proc/uptime and the information is updated by a timer.
Resource monitoring and interpretation	Threshold-based system health (Normal, Warning, High).

Important: The project demonstrates how an operating system's resource information can be observed and interpreted. It does not itself implement process creation using fork() or exec(), threads, semaphores, shared memory or inter-process communication, and these concepts are therefore not claimed as implemented.

CHAPTER 8 CONCLUSION

8.1 Summary of the Project

The Linux Resource Monitoring Tool (submitted as the Linux System Information and Resource Monitoring Tool) is a C++ application with a GTK4 graphical interface that monitors the major resources of a Linux system. It collects CPU, memory, swap and uptime information from the `/proc` virtual filesystem, obtains process and disk information from the Linux utilities `ps` and `df`, and obtains the number of CPU cores with `sysconf()`. The collected data is converted into percentages and readable values, and displayed on a dashboard that also includes CPU and memory history graphs, the top CPU and memory consumers, a simple system health indicator, automatic refreshing, a Refresh button and an About Project dialog.

The project was developed and tested in Ubuntu through WSL, compiled with `g++`, and gave the team practical experience in connecting Operating Systems theory with an actual Linux system.

8.2 Objectives Achieved

Table 8.1: Objectives and Their Achievement

Objective	How it was Achieved
Study Linux information sources	<code>/proc/stat</code> , <code>/proc/meminfo</code> , <code>/proc/uptime</code> , <code>ps</code> , <code>df</code> and <code>sysconf()</code> were studied and used.
Calculate meaningful usage values	CPU, memory, swap and disk usage were calculated as percentages.
Identify heavy processes	Top CPU consumer and top memory consumer are identified from the <code>ps</code> output.
Graphical dashboard	A GTK4 dashboard with cards, graphs and buttons was developed.
Usage history	CPU and memory history graphs were implemented.
Automatic refresh	A GTK4 timer updates the dashboard periodically, and a Refresh button updates it on demand.
Simple interpretation of usage	System Health with Normal, Warning and High thresholds was provided.
Demonstrate OS concepts	CPU, memory, process and file system concepts are demonstrated with real data.

8.3 Results Achieved

- A working GTK4 desktop monitoring tool for Linux was developed using C++.
- The tool displays CPU, memory, disk and swap usage, total memory, CPU cores, uptime and running process count.
- The tool identifies the top CPU consumer and the top memory consumer.
- Two history graphs show the recent behaviour of CPU and memory usage.
- The dashboard refreshes automatically and can also be refreshed manually.
- The tool was compiled with the command `g++ main.cpp -o monitor $(pkg-config --cflags --libs gtk4)` and executed with `./monitor` in Ubuntu through WSL.

8.4 Skills and Learning Outcomes

Through this project, the team members gained the following knowledge and skills:

- Understanding of CPU utilisation, memory management, swap, processes and file systems in Linux.
- Knowledge of the structure and use of the `/proc` virtual filesystem.
- Use of Linux utilities such as `ps` and `df` and their execution from a C++ program using `popen()` and `pclose()`.
- Use of the `sysconf()` interface to obtain system configuration.
- Event-driven GUI programming with GTK4, including timers, buttons, dialogs and drawing areas.
- Drawing graphs using Cairo/GTK drawing functionality.
- Working in the Ubuntu/WSL environment and compiling programs with `g++` and `pkg-config`.
- Teamwork, task division, testing and technical documentation.

8.5 Future Enhancements

The current version provides the features described in this report. The following enhancements are **possible future work** and are not implemented in the present version:

Table 8.2: Possible Future Enhancements

Enhancement	Description
More detailed graphs	Longer history, additional graphs for disk or swap, and graphs with scales and labels.
Resource usage alerts	Notifications when CPU, memory or disk usage crosses a limit.
Process filtering	Searching and filtering processes by name, PID or resource usage.
Process control	Controlling processes, such as changing priority or terminating a process, with suitable confirmation.
CPU temperature monitoring	Showing CPU temperature where the hardware and system support it.
Network monitoring	Showing network usage and statistics.
Historical data storage	Saving monitoring data for later analysis.
Exporting monitoring reports	Exporting the collected information as a report.

Current limitations: The process CPU percentage obtained from `ps` is an average over the lifetime of a process and not an instantaneous value; the tool shows only the top CPU and memory consumers and not a complete process list; and the information is limited to the Linux environment in which it runs, which in the case of WSL is the Ubuntu environment and not the Windows host.

CHAPTER 9

REFERENCES

The following documentation and books were referred to during the study and development of the project.

- [1] Linux man-pages project, proc(5) – process information pseudo-filesystem. Available at: <https://man7.org/linux/man-pages/man5/proc.5.html>
- [2] The Linux Kernel documentation, The /proc Filesystem. Available at: <https://docs.kernel.org/filesystems/proc.html>
- [3] Linux man-pages project, ps(1), df(1), sysconf(3), popen(3) and pclose(3) manual pages. Available at: <https://man7.org/linux/man-pages/>
- [4] GNU Compiler Collection (GCC) Documentation. Available at: <https://gcc.gnu.org/onlinedocs/>
- [5] GTK Project, GTK4 Documentation. Available at: <https://docs.gtk.org/gtk4/>
- [6] Cairo Graphics, Cairo Documentation. Available at: <https://www.cairographics.org/documentation/>
- [7] Ubuntu Documentation. Available at: <https://ubuntu.com/>
- [8] Microsoft Learn, Windows Subsystem for Linux Documentation. Available at: <https://learn.microsoft.com/windows/wsl/>
- [9] cppreference.com, C++ Reference. Available at: <https://en.cppreference.com/>
- [10] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.
- [11] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2014.
- [12] M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.